

Résumé Complet Cassandra CQL

CRUD, Requêtes, Filtrage

AmineGR03

1^{er} février 2026

Table des matières

1	Création de Structure	2
1.1	Keyspace	2
1.2	Table	2
1.3	Table avec Clustering Key	2
1.4	Table avec Partition Key Composite	2
2	Opérations CRUD	3
2.1	Create (INSERT)	3
2.1.1	Insertion simple	3
2.1.2	Insertion avec UUID spécifique	3
2.1.3	Insertion avec TTL (Time To Live)	3
2.1.4	Insertion avec TIMESTAMP	3
2.1.5	Insertion conditionnelle (IF NOT EXISTS)	3
2.2	Read (SELECT)	3
2.2.1	Sélection de tous les champs	3
2.2.2	Sélection de champs spécifiques	3
2.2.3	Sélection avec WHERE	4
2.2.4	Sélection avec LIMIT	4
2.2.5	Sélection avec COUNT	4
2.2.6	Sélection avec DISTINCT	4
2.3	Update (UPDATE)	4
2.3.1	Mise à jour simple	4
2.3.2	Mise à jour avec TTL	4
2.3.3	Mise à jour conditionnelle (IF)	4
2.3.4	Mise à jour avec IF EXISTS	4
2.3.5	Opérations sur Collections	5
2.4	Delete (DELETE)	5
2.4.1	Suppression d'un document	5
2.4.2	Suppression de champs spécifiques	5
2.4.3	Suppression d'éléments dans une collection	5
2.4.4	Suppression conditionnelle	5
2.4.5	TRUNCATE (vider une table)	5
3	Filtrage et Requêtes	6
3.1	Opérateurs de Comparaison	6
3.2	Opérateur IN	6
3.3	Opérateurs Logiques	6
3.4	Opérateur LIKE	6

3.5	ALLOW FILTERING	6
3.6	Requêtes sur Clustering Columns	7
4	Collections	8
4.1	List	8
4.2	Set	8
4.3	Map	8
5	Index	10
5.1	Index Secondaire	10
5.2	Index sur Collection	10
5.3	Index sur Clé de Map	10
5.4	Index sur Clé de Map (KEYS)	10
5.5	Index sur Valeur de Map (VALUES)	10
5.6	Index sur Entrées de Map (ENTRIES)	10
6	Types de Données	11
6.1	Types Primitifs	11
6.2	Types Temporels	11
6.3	Types Collections	11
6.4	Types Utilisateur (UDT)	12
7	Fonctions	13
7.1	Fonctions de Conversion	13
7.2	Fonctions Temporelles	13
7.3	Fonctions sur Collections	13
8	BATCH	14
8.1	Batch Simple	14
8.2	Batch avec Timestamp	14
8.3	Batch avec TTL	14
9	Requêtes Avancées	15
9.1	ORDER BY	15
9.2	GROUP BY	15
9.3	Requêtes avec Token	15
9.4	Requêtes sur Collections	15
10	Modification de Structure	16
10.1	ALTER TABLE	16
10.2	DROP	16
11	Résumé des Opérations Essentielles	17
11.1	CRUD	17
11.2	Opérateurs de Filtrage	17
11.3	Collections	17
11.4	Points Importants	17

Création de Structure

Keyspace

```
1 CREATE KEYSPACE ma_base
2 WITH REPLICATION = {
3     'class': 'SimpleStrategy',
4     'replication_factor': 3
5 };
6
7 USE ma_base;
```

Table

```
1 CREATE TABLE utilisateurs (
2     id UUID PRIMARY KEY,
3     nom TEXT,
4     email TEXT,
5     age INT,
6     ville TEXT,
7     date_creation TIMESTAMP
8 );
```

Table avec Clustering Key

```
1 CREATE TABLE commandes (
2     client_id UUID,
3     date_commande TIMESTAMP,
4     produit TEXT,
5     quantite INT,
6     prix DECIMAL,
7     PRIMARY KEY (client_id, date_commande, produit)
8 );
```

Table avec Partition Key Composite

```
1 CREATE TABLE articles (
2     categorie TEXT,
3     sous_categorie TEXT,
4     id UUID,
5     titre TEXT,
6     prix DECIMAL,
7     PRIMARY KEY ((categorie, sous_categorie), id)
8 );
```

Opérations CRUD

Create (INSERT)

Insertion simple

```
1 INSERT INTO utilisateurs (id, nom, email, age, ville)
2 VALUES (uuid(), 'Dupont', 'dupont@email.com', 30, 'Paris');
```

Insertion avec UUID spécifique

```
1 INSERT INTO utilisateurs (id, nom, email, age, ville)
2 VALUES (
3     a1b2c3d4-e5f6-7890-abcd-ef1234567890,
4     'Martin',
5     'martin@email.com',
6     25,
7     'Lyon'
8 );
```

Insertion avec TTL (Time To Live)

```
1 INSERT INTO utilisateurs (id, nom, email)
2 VALUES (uuid(), 'Bernard', 'bernard@email.com')
3 USING TTL 3600; -- Expire apr s 1 heure
```

Insertion avec TIMESTAMP

```
1 INSERT INTO utilisateurs (id, nom, date_creation)
2 VALUES (uuid(), 'Durand', toTimestamp(now()));
```

Insertion conditionnelle (IF NOT EXISTS)

```
1 INSERT INTO utilisateurs (id, nom, email)
2 VALUES (a1b2c3d4-e5f6-7890-abcd-ef1234567890, 'Dupont', 'dupont@email.
3 com')
4 IF NOT EXISTS;
```

Read (SELECT)

Sélection de tous les champs

```
1 SELECT * FROM utilisateurs;
```

Sélection de champs spécifiques

```
1 SELECT id, nom, email FROM utilisateurs;
```

Sélection avec WHERE

```
1 SELECT * FROM utilisateurs WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;
```

Sélection avec LIMIT

```
1 SELECT * FROM utilisateurs LIMIT 10;
```

Sélection avec COUNT

```
1 SELECT COUNT(*) FROM utilisateurs;
```

Sélection avec DISTINCT

```
1 SELECT DISTINCT ville FROM utilisateurs;
```

Update (UPDATE)

Mise à jour simple

```
1 UPDATE utilisateurs
2 SET age = 31, ville = 'Lyon'
3 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;
```

Mise à jour avec TTL

```
1 UPDATE utilisateurs
2 USING TTL 7200
3 SET email = 'nouveau@email.com'
4 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;
```

Mise à jour conditionnelle (IF)

```
1 UPDATE utilisateurs
2 SET age = 32
3 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890
4 IF age = 31;
```

Mise à jour avec IF EXISTS

```
1 UPDATE utilisateurs
2 SET age = 33
3 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890
4 IF EXISTS;
```

Opérations sur Collections

```
1 -- Ajouter une liste
2 UPDATE utilisateurs
3 SET hobbies = hobbies + ['lecture']
4 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;
5
6 -- Retirer d'une liste
7 UPDATE utilisateurs
8 SET hobbies = hobbies - ['sport']
9 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;
10
11 -- Ajouter un set
12 UPDATE utilisateurs
13 SET tags = tags + {'important'}
14 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;
15
16 -- Mettre jour une map
17 UPDATE utilisateurs
18 SET adresse = {'rue': '123 Main St', 'ville': 'Paris'}
19 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;
```

Delete (DELETE)

Suppression d'un document

```
1 DELETE FROM utilisateurs
2 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;
```

Suppression de champs spécifiques

```
1 DELETE email, age FROM utilisateurs
2 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;
```

Suppression d'éléments dans une collection

```
1 DELETE hobbies[0] FROM utilisateurs
2 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;
```

Suppression conditionnelle

```
1 DELETE FROM utilisateurs
2 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890
3 IF age = 30;
```

TRUNCATE (vider une table)

```
1 TRUNCATE utilisateurs;
```

Filtrage et Requêtes

Opérateurs de Comparaison

```
1 -- galit
2 SELECT * FROM utilisateurs WHERE age = 30;
3
4 -- Plus grand que
5 SELECT * FROM utilisateurs WHERE age > 25;
6
7 -- Plus grand ou gal
8 SELECT * FROM utilisateurs WHERE age >= 25;
9
10 -- Plus petit que
11 SELECT * FROM utilisateurs WHERE age < 40;
12
13 -- Plus petit ou gal
14 SELECT * FROM utilisateurs WHERE age <= 40;
15
16 -- Différent de
17 SELECT * FROM utilisateurs WHERE age != 30;
```

Opérateur IN

```
1 SELECT * FROM utilisateurs
2 WHERE ville IN ('Paris', 'Lyon', 'Marseille');
```

Opérateurs Logiques

```
1 -- ET
2 SELECT * FROM utilisateurs
3 WHERE age > 25 AND ville = 'Paris';
4
5 -- OU
6 SELECT * FROM utilisateurs
7 WHERE age < 30 OR ville = 'Lyon';
```

Opérateur LIKE

```
1 -- Commence par
2 SELECT * FROM utilisateurs
3 WHERE nom LIKE 'Dup%';
4
5 -- Contient
6 SELECT * FROM utilisateurs
7 WHERE email LIKE '%@gmail.com';
```

ALLOW FILTERING

```
1 -- Permet de filtrer sur des colonnes non-indexées
2 -- ATTENTION: Peut être très coûteux en performance
3 SELECT * FROM utilisateurs
```

```
4 WHERE ville = 'Paris' AND age > 25
5 ALLOW FILTERING;
```

Requêtes sur Clustering Columns

```
1 -- Requ te sur partition key
2 SELECT * FROM commandes
3 WHERE client_id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;
4
5 -- Requ te avec clustering key
6 SELECT * FROM commandes
7 WHERE client_id = a1b2c3d4-e5f6-7890-abcd-ef1234567890
8 AND date_commande > '2024-01-01';
9
10 -- Requ te avec range sur clustering key
11 SELECT * FROM commandes
12 WHERE client_id = a1b2c3d4-e5f6-7890-abcd-ef1234567890
13 AND date_commande >= '2024-01-01'
14 AND date_commande <= '2024-12-31';
```


Collections

List

```

1 CREATE TABLE utilisateurs (
2     id UUID PRIMARY KEY,
3     nom TEXT,
4     hobbies LIST<TEXT>
5 );
6
7 -- Insertion
8 INSERT INTO utilisateurs (id, nom, hobbies)
9 VALUES (uuid(), 'Dupont', ['lecture', 'sport', 'musique']);
10
11 -- Mise à jour
12 UPDATE utilisateurs
13 SET hobbies = hobbies + ['cinema']
14 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;
15
16 -- Retirer un élément
17 UPDATE utilisateurs
18 SET hobbies = hobbies - ['sport']
19 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;

```

Set

```

1 CREATE TABLE articles (
2     id UUID PRIMARY KEY,
3     titre TEXT,
4     tags SET<TEXT>
5 );
6
7 -- Insertion
8 INSERT INTO articles (id, titre, tags)
9 VALUES (uuid(), 'Article 1', {'tech', 'programming', 'database'});
10
11 -- Ajouter un élément
12 UPDATE articles
13 SET tags = tags + {'important'}
14 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;

```

Map

```

1 CREATE TABLE utilisateurs (
2     id UUID PRIMARY KEY,
3     nom TEXT,
4     adresse MAP<TEXT, TEXT>
5 );
6
7 -- Insertion
8 INSERT INTO utilisateurs (id, nom, adresse)
9 VALUES (uuid(), 'Dupont', {
10     'rue': '123 Main St',
11     'ville': 'Paris',
12     'codePostal': '75001'

```

```
13 });  
14  
15 -- Mise      jour  
16 UPDATE utilisateurs  
17 SET adresse['ville'] = 'Lyon'  
18 WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;
```

Index

Index Secondaire

```
1 CREATE INDEX idx_ville ON utilisateurs (ville);
2
3 -- Utilisation
4 SELECT * FROM utilisateurs WHERE ville = 'Paris';
```

Index sur Collection

```
1 CREATE INDEX idx_tags ON articles (tags);
2
3 -- Recherche dans un set
4 SELECT * FROM articles WHERE tags CONTAINS 'tech';
```

Index sur Clé de Map

```
1 CREATE INDEX idx_ville_adresse ON utilisateurs (adresse);
2
3 -- Recherche
4 SELECT * FROM utilisateurs WHERE adresse['ville'] = 'Paris';
```

Index sur Clé de Map (KEYS)

```
1 CREATE INDEX idx_adresse_keys ON utilisateurs (KEYS(adresse));
```

Index sur Valeur de Map (VALUES)

```
1 CREATE INDEX idx_adresse_values ON utilisateurs (VALUES(adresse));
```

Index sur Entrées de Map (ENTRIES)

```
1 CREATE INDEX idx_adresse_entries ON utilisateurs (ENTRIES(adresse));
```

Types de Données

Types Primitifs

```
1 -- Text
2 nom TEXT
3
4 -- Integer
5 age INT
6
7 -- Big Integer
8 grand_nombre BIGINT
9
10 -- Decimal
11 prix DECIMAL
12
13 -- Float
14 valeur FLOAT
15
16 -- Double
17 valeur_double DOUBLE
18
19 -- Boolean
20 est_actif BOOLEAN
21
22 -- UUID
23 id UUID
24
25 -- TimeUUID
26 timestamp TIMEUUID
```

Types Temporels

```
1 -- Date
2 date_naissance DATE
3
4 -- Time
5 heure TIME
6
7 -- Timestamp
8 date_creation TIMESTAMP
```

Types Collections

```
1 -- Liste
2 hobbies LIST<TEXT>
3
4 -- Set
5 tags SET<INT>
6
7 -- Map
8 adresse MAP<TEXT, TEXT>
```

Types Utilisateur (UDT)

```
1 -- Cr ation d'un type
2 CREATE TYPE adresse_type (
3     rue TEXT,
4     ville TEXT,
5     code_postal TEXT
6 );
7
8 -- Utilisation
9 CREATE TABLE utilisateurs (
10     id UUID PRIMARY KEY,
11     nom TEXT,
12     adresse adresse_type
13 );
```

Fonctions

Fonctions de Conversion

```
1 -- UUID vers String
2 SELECT toText(id) FROM utilisateurs;
3
4 -- String vers UUID
5 SELECT toUUID('a1b2c3d4-e5f6-7890-abcd-ef1234567890') FROM utilisateurs;
6
7 -- Timestamp
8 SELECT toTimestamp(now()) FROM utilisateurs;
```

Fonctions Temporelles

```
1 -- Date actuelle
2 SELECT now() FROM utilisateurs;
3
4 -- Timestamp Unix
5 SELECT toUnixTimestamp(date_creation) FROM utilisateurs;
6
7 -- Depuis Unix timestamp
8 SELECT toTimestamp(1234567890) FROM utilisateurs;
```

Fonctions sur Collections

```
1 -- Taille d'une liste
2 SELECT size(hobbies) FROM utilisateurs;
3
4 -- Taille d'un set
5 SELECT size(tags) FROM articles;
6
7 -- Taille d'une map
8 SELECT size(adresse) FROM utilisateurs;
```

BATCH

Batch Simple

```
1 BEGIN BATCH
2   INSERT INTO utilisateurs (id, nom, email)
3   VALUES (uuid(), 'Dupont', 'dupont@email.com');
4
5   INSERT INTO utilisateurs (id, nom, email)
6   VALUES (uuid(), 'Martin', 'martin@email.com');
7
8   UPDATE utilisateurs
9   SET age = 30
10  WHERE id = a1b2c3d4-e5f6-7890-abcd-ef1234567890;
11 APPLY BATCH;
```

Batch avec Timestamp

```
1 BEGIN BATCH
2   USING TIMESTAMP 1234567890
3   INSERT INTO utilisateurs (id, nom)
4   VALUES (uuid(), 'Dupont');
5 APPLY BATCH;
```

Batch avec TTL

```
1 BEGIN BATCH
2   USING TTL 3600
3   INSERT INTO utilisateurs (id, nom)
4   VALUES (uuid(), 'Dupont');
5 APPLY BATCH;
```

Requêtes Avancées

ORDER BY

```
1 -- Tri sur clustering column
2 SELECT * FROM commandes
3 WHERE client_id = a1b2c3d4-e5f6-7890-abcd-ef1234567890
4 ORDER BY date_commande DESC;
```

GROUP BY

```
1 -- Groupement (nécessite ALLOW FILTERING)
2 SELECT ville, COUNT(*) FROM utilisateurs
3 GROUP BY ville
4 ALLOW FILTERING;
```

Requêtes avec Token

```
1 -- Requête par range de tokens
2 SELECT * FROM utilisateurs
3 WHERE token(id) > token(minTimeuuid('2024-01-01'))
4 AND token(id) <= token(maxTimeuuid('2024-12-31'));
```

Requêtes sur Collections

```
1 -- Contient dans une liste
2 SELECT * FROM utilisateurs
3 WHERE hobbies CONTAINS 'lecture';
4
5 -- Contient dans un set
6 SELECT * FROM articles
7 WHERE tags CONTAINS 'tech';
8
9 -- Clé existe dans une map
10 SELECT * FROM utilisateurs
11 WHERE adresse CONTAINS KEY 'ville';
12
13 -- Valeur existe dans une map
14 SELECT * FROM utilisateurs
15 WHERE adresse CONTAINS 'Paris';
```


Modification de Structure

ALTER TABLE

```
1 -- Ajouter une colonne
2 ALTER TABLE utilisateurs ADD telephone TEXT;
3
4 -- Supprimer une colonne
5 ALTER TABLE utilisateurs DROP telephone;
6
7 -- Modifier le type d'une colonne
8 ALTER TABLE utilisateurs ALTER telephone TYPE TEXT;
```

DROP

```
1 -- Supprimer une table
2 DROP TABLE utilisateurs;
3
4 -- Supprimer un keyspace
5 DROP KEYSPACE ma_base;
6
7 -- Supprimer un index
8 DROP INDEX idx_ville;
```

Résumé des Opérations Essentielles

CRUD

Opération	Commande
Insert	INSERT INTO ... VALUES ...
Read	SELECT ... FROM ... WHERE ...
Update	UPDATE ... SET ... WHERE ...
Delete	DELETE FROM ... WHERE ...

Opérateurs de Filtrage

Opérateur	Description
=	Égalité
>	Plus grand que
>=	Plus grand ou égal
<	Plus petit que
<=	Plus petit ou égal
!=	Différent de
IN	Dans une liste
LIKE	Pattern matching
CONTAINS	Contient (collections)
CONTAINS KEY	Clé existe (map)

Collections

Type	Description
LIST<T>	Liste ordonnée
SET<T>	Ensemble non ordonné
MAP<K, V>	Dictionnaire clé-valeur

Points Importants

- **Partition Key** : Détermine sur quel nœud les données sont stockées
- **Clustering Key** : Détermine l'ordre des données dans une partition
- **ALLOW FILTERING** : Permet de filtrer sur des colonnes non-indexées (coûteux)
- **TTL** : Permet d'expirer automatiquement les données
- **BATCH** : Permet d'exécuter plusieurs opérations atomiquement
- **IF EXISTS / IF NOT EXISTS** : Conditions pour INSERT/UPDATE/DELETE